Stream Savvy

AP23110010994

AP23110010693

AP23110010912

AP23110010972

Introduction:-

Step into the world of limitless entertainment with **Stream Savvy**, your ultimate destination for movies and TV shows. Designed with the power of **React.js**, Stream Savvy offers a fluid, immersive, and visually stunning experience for entertainment enthusiasts. Whether you are looking for the latest blockbusters, timeless classics, or trending TV series, Stream Savvy curates it all in one sleek interface.

Built for the modern viewer, this application seamlessly blends high-performance functionality with an intuitive user interface. Say goodbye to endless scrolling without direction—Stream Savvy brings the content you love right to your fingertips, optimized for desktops, tablets, and smartphones alike.

One of the key features of this app is the **watchlist**, where users can mark and save movies or shows they wish to watch later. This feature makes it easier for users to maintain their personalized collection without having to search repeatedly. The app functions as a **personal entertainment library**, allowing individuals to organize and track their favourite titles efficiently.

Although this application does not host actual streaming video content, it simulates the browsing and selection experience of a real OTT platform. It focuses on providing smooth navigation, visually appealing interfaces, and structured information for every title. The project also emphasizes good design principles, intuitive UI/UX, and efficient data management to ensure a seamless user experience.

Scenario-Based Intro:-

Imagine it’s Friday night. You’ve had a long week, and you’re ready to unwind. You grab your snacks, settle onto the couch, and open **Stream Savvy**.

Instead of being overwhelmed by cluttered menus, you are greeted by a clean, cinematic homepage showcasing the top trending movies of the week. You type "Action" into the search bar, and instantly, a curated list of adrenaline-pumping films appears. You click on a poster, read the synopsis, check the rating, and hit "Play" (or

add it to your watchlist). The experience is smooth, fast, and effortless. This is the power of Stream Savvy making your entertainment choices smart and simple.

Target Audience

Stream Savvy is tailored for a wide range of users, including:

- **Movie Buffs:** Individuals who love keeping up with the latest releases and hidden gems.
- **Binge-Watchers:** TV show enthusiasts looking for their next series addiction.
- **Casual Viewers:** People who want a quick and easy way to find something to watch without hassle.

Project Goals and Objectives

The primary goal of Stream Savvy is to provide a central hub for entertainment discovery.

- **User-Friendly Interface:** Create a visually appealing and easy-to-navigate layout.
- **Seamless Discovery:** Implement robust search and filtering capabilities.
- **Responsive Design:** Ensure the application looks and works perfectly on all screen sizes.
- **Modern Tech Stack:** Utilize **React.js** and **Vite** for fast performance and **Tailwind CSS** for modern styling.

Key Features

- **Trending Feed:** A dynamic homepage displaying currently popular movies and shows.
- **Smart Search:** Real-time search functionality to find titles instantly.
- **Detailed Views:** Comprehensive information pages for each title, including plot summaries, ratings, and cast info.
- **Responsive Layout:** A fluid grid system that adapts to mobile and desktop screens.

PRE-REQUISITES:-

Here are the key prerequisites for developing a frontend application using React.js:

- **Node.js and npm**:

  Node.js is a powerful JavaScript runtime environment that allows you to run JavaScript code on the local environment. It provides a scalable and efficient platform for building network applications.

  Install Node.js and npm on your development machine, as they are required to run JavaScript on the server-side.

  - Download: https://nodejs.org/en/download/
  - Installation instructions: https://nodejs.org/en/download/package-manager/

- **React.js**:

  React.js is a popular JavaScript library for building user interfaces. It enables developers to create interactive and reusable UI components, making it easier to build dynamic and responsive web applications.

  Install React.js, a JavaScript library for building user interfaces.

  - Create a new React app:

    npm create vite@latest

    Enter and then type project-name and select preferred frameworks and then enter

  - Navigate to the project directory:

    cd project-name

    npm install

  - Running the React App:

    With the React app created, you can now start the development server and see your React application in action.

  - Start the development server:

    npm run dev

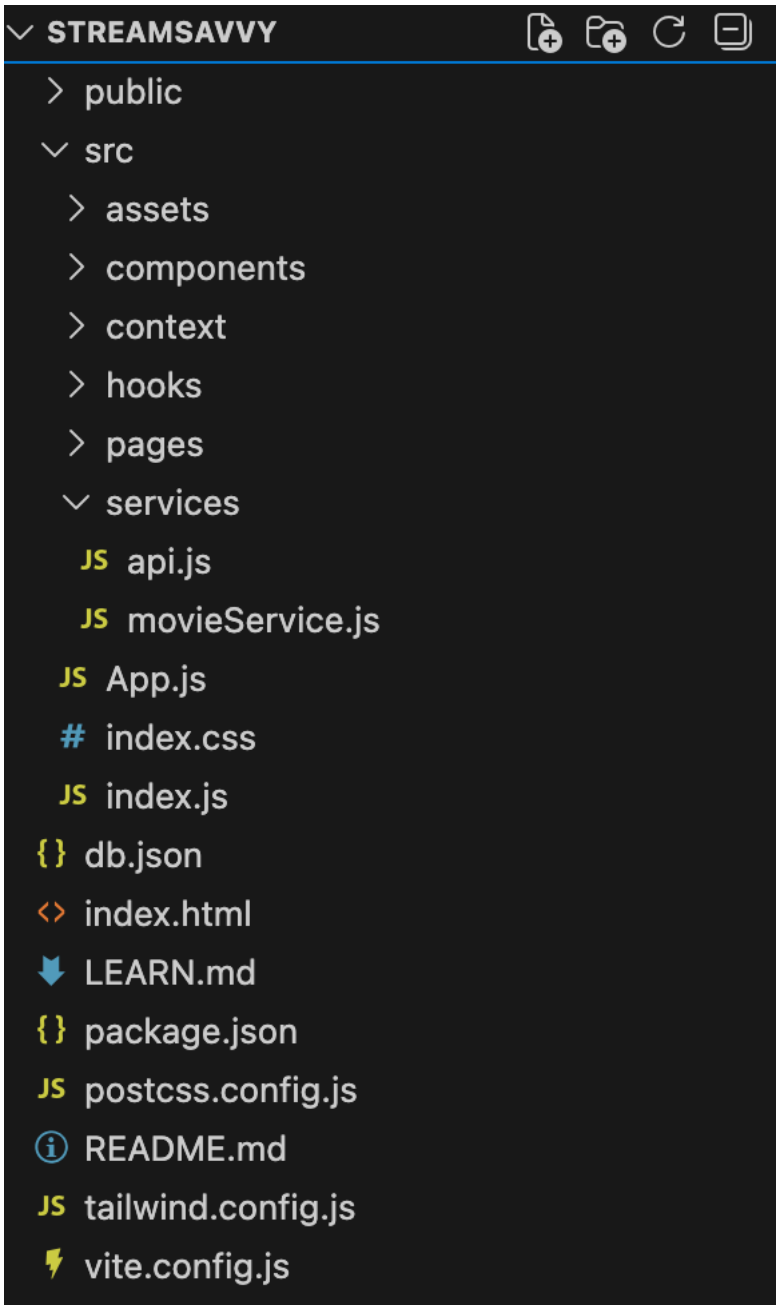
    This command launches the development server, and you can access your React app at http://localhost:5173 in your web browser.

- **HTML, CSS, and JavaScript**: Basic knowledge of HTML for creating the structure of your app, CSS for styling, and JavaScript for client-side interactivity is essential.

- **Version Control**: Use Git for version control, enabling collaboration and tracking changes throughout the development process. Platforms like GitHub or Bitbucket can host your repository.
    - Git: Download and installation instructions can be found at: https://git-scm.com/downloads

- **Development Environment**: Choose a code editor or Integrated Development Environment (IDE) that suits your preferences, such as Visual Studio Code, Sublime Text, or WebStorm.
    - VisualStudioCode:Downloadfrom https://code.visualstudio.com/download
    - SublimeText: Download from https://www.sublimetext.com/download
    - WebStorm: Download from https://www.jetbrains.com/webstorm/download

Project structure:



The project structure may vary depending on the specific library, framework, programming language, or development approach used. It's essential to organize the files and directories in a logical and consistent manner to improve code maintainability and collaboration among developers.

app/app.component.css, src/app/app.component: These files are part of the main AppComponent, which serves as the root component for the React app. The component handles the overall layout and includes the router outlet for loading different components based on the current route.

PROJECT FLOW:-

Project demo:

Before starting to work on this project, let's see the demo.

Demo link:

https://drive.google.com/file/d/1xBUffe4dDobCSZ-_ZqZklHeN5oDbvIGS/view?usp=sharing

Use the code in:

https://drive.google.com/file/d/1X-uAJBWiGz9Psy80G49Y0MdyIOXMZDpq/view?usp=sharing

Milestone 1: Project Setup and Configuration:

1. Install required tools and software:

- **Installation of required tools**:

    1. Open the project folder to install necessary tools
       In this project, we use:
        - React Js
        - React Icons
        - Material-ui
        - React-codemirror2

- For further reference, use the following resources
    - https://react.dev/learn/installation
    - https://react-bootstrap-v4.netlify.app/getting-started/introduction/

Milestone 2: Web Development:

1. Setup React Application:

• Create React application.

• Configure Routing.

• Install required libraries.

App.js component:

App.js

src > App.js > ...

```jsx
import React from 'react';
import { BrowserRouter, Routes, Route } from 'react-router-dom';

// Contexts
import { ThemeProvider } from './context/ThemeContext';
import { PlayerProvider } from './context/PlayerContext';
import { AuthProvider } from './context/AuthContext';

// Components
import Navbar from './components/layout/Navbar';
import MiniPlayer from './components/player/MiniPlayer';
import { ToastContainer } from 'react-toastify';
import 'react-toastify/dist/ReactToastify.css';

// Pages
import Home from './pages/Home';
import Browse from './pages/Browse';
import MovieDetails from './pages/MovieDetails';
import AddEditMovie from './pages/AddEditMovie';
import Login from './pages/Login';
import MyList from './pages/MyList'; // <--- IMPORT MYLIST

function App() {
  return (
    <ThemeProvider>
      <AuthProvider>
        <PlayerProvider>
          <BrowserRouter>
            <div className="min-h-screen font-sans bg-gray-100 dark:bg-dark text-gray-900 dark:text-white transition-colors duration-300">

              <Navbar />

              <Routes>
                <Route path="/" element={<Home />} />
                <Route path="/browse" element={<Browse />} />
                <Route path="/movie/:id" element={<MovieDetails />} />
                <Route path="/add" element={<AddEditMovie />} />
                <Route path="/edit-movie/:id" element={<AddEditMovie />} />
                <Route path="/login" element={<Login />} />

                {/* MYLIST ROUTE UPDATED */}
                <Route path="/mylist" element={<MyList />} />
              </Routes>

              <MiniPlayer />
              <ToastContainer position="bottom-right" theme="dark" />
            </div>
          </BrowserRouter>
        </PlayerProvider>
      </AuthProvider>
    </ThemeProvider>
  );
}
```

Code Description:-

● **ThemeProvider from ./context/ThemeContext:**A context provider that manages the application's theme (light/dark mode) and makes theme values available across all components.

● **AuthProvider from ./context/AuthContext:**A global context provider responsible for authentication state (user login, logout, session management).

● **PlayerProvider from ./context/PlayerContext:**Manages the video player's global state, such as current playing movie, controls, and mini-player visibility.

● **BrowserRouter, Routes, and Route from react-router-dom:**Used to create the navigation system of the app, allowing users to visit multiple pages like Home, Browse, Movie Details, Login, My List, etc.

● **Navbar component:**Renders the top navigation bar for accessing different pages.

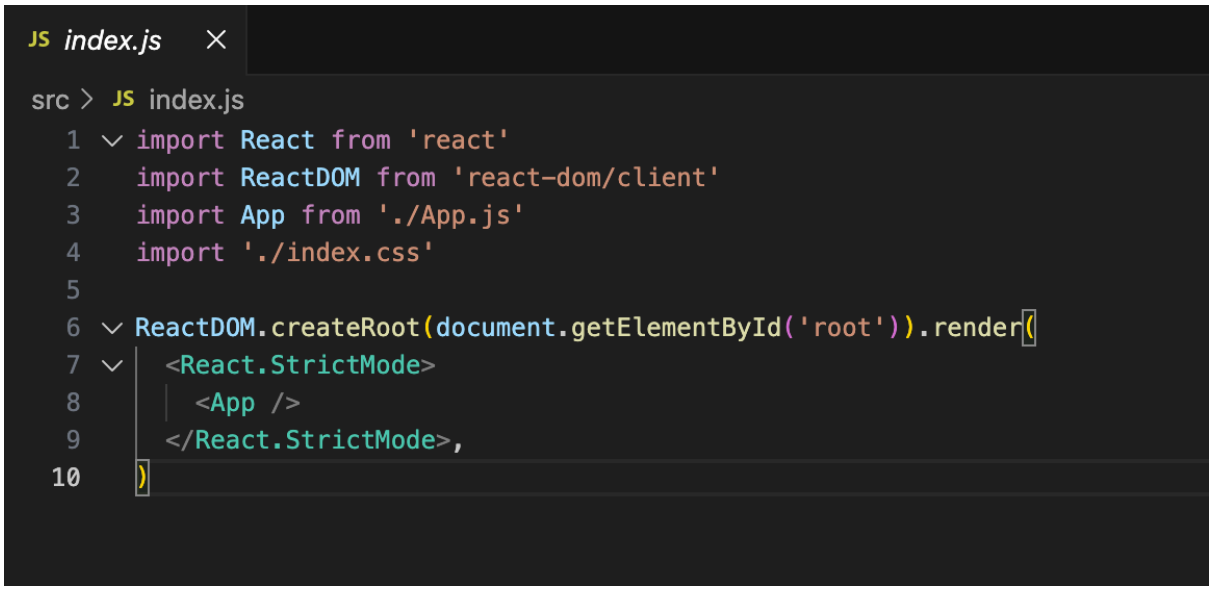
● **MiniPlayer component:**Displays a small floating video player that remains visible across pages.

● **ToastContainer from react-toastify:**Shows notifications such as "Added to List", "Login Successful", etc.

● **Pages imported:**

- **Home** – Displays the landing page.
- **Browse** – Shows movies categorized by genres.
- **MovieDetails** – Shows details of a selected movie based on its ID.
- **AddEditMovie** – Used for adding or editing a movie.
- **Login** – Manages user login.
- **MyList** – Displays the user’s saved movies/watchlist.

Data Provider:-



Code Description:-

● React and ReactDOM Imports:

- React is imported to use JSX and component features.
- ReactDOM is imported to render the React application into the browser DOM.

● App from ./App.js:

The main root component of the application. All pages, routes, and providers are contained inside the App component.

● ./index.css:

A global CSS file that contains styling rules applied across the entire application.

● ReactDOM.createRoot(...):

Creates a React root using the new React 18 API, enabling improved performance and concurrent rendering.

- **document.getElementById('root'):**

Selects the HTML element with id "root" from **index.html**.
This is where the whole React UI will be mounted.

Code Component:-

```jsx
import React, { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';

const Login = () => {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const { login } = useAuth();
  const navigate = useNavigate();

  const handleSubmit = (e) => {
    e.preventDefault();
    if (login(email, password)) {
      navigate('/'); // Redirect to home on successful login
    }
  };

  return (
    <div className="pt-24 min-h-screen flex justify-center items-center px-4">
      <form onSubmit={handleSubmit} className="w-full max-w-sm bg-white dark:bg-grayd p-8 rounded-lg shadow-xl">
        <h2 className="text-3xl font-bold mb-6 text-center text-primary">Sign In</h2>
        <div className="space-y-4">
          <input
            type="email"
            placeholder="Email (test@test.com)"
            required
            value={email}
            onChange={(e) => setEmail(e.target.value)}
            className="w-full p-3 rounded-md border dark:bg-gray-700 dark:border-gray-600 dark:text-white focus:ring-2 focus:ring-primary"
          />
          <input
            type="password"
            placeholder="Password (123)"
            required
            value={password}
            onChange={(e) => setPassword(e.target.value)}
            className="w-full p-3 rounded-md border dark:bg-gray-700 dark:border-gray-600 dark:text-white focus:ring-2 focus:ring-primary"
          />
          <button
            type="submit"
            className="w-full bg-primary text-white py-3 rounded-md font-bold hover:bg-red-700 transition"
          >
            Log In
          </button>
        </div>
      </form>
    </div>
  );
};

export default Login;
```

Code Description:-

- **React** is imported so we can write JSX and use React features.
- **ReactDOM** is imported to render React components into the webpage.
- The main app component **App** is imported from ./App.
- A global CSS file **styles.css** is imported for styling.
- ReactDOM.createRoot() creates the main React rendering root.
- document.getElementById('root') selects the HTML element where the app will load.

- The <React.StrictMode> wrapper helps catch errors during development.
- The <App /> component is rendered inside StrictMode.
- This file is the **entry point** that starts the entire React application.

Code-Editor Component:-

```jsx
import React, { useEffect, useState } from 'react';
import { getFavorites, getMovieById } from '../services/movieService';
import MovieCard from '../components/movies/MovieCard';
import { toast } from 'react-toastify';
import { FaHeart } from 'react-icons/fa';

const MyList = () => {
  const [favoriteMovies, setFavoriteMovies] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetchFavorites();
  }, []);

  const fetchFavorites = async () => {
    try {
      // 1. Get the list of favorite entries
      const favRes = await getFavorites();
      const favoriteEntries = favRes.data;

      if (favoriteEntries.length > 0) {
        // 2. Fetch the full movie data for each favorite entry
        // We use Promise.all to fetch them all in parallel
        const moviePromises = favoriteEntries.map(entry => getMovieById(entry.movieId));
        const movieResults = await Promise.all(moviePromises);

        // 3. Extract the movie data
        const movies = movieResults.map(res => res.data);
        setFavoriteMovies(movies);
      } else {
        setFavoriteMovies([]);
      }

    } catch (error) {
      console.error(error);
      toast.error("Could not load My List.");
    } finally {
      setLoading(false);
    }
  };

  if (loading) return <div className="pt-24 text-center dark:text-white text-xl">Loading My List...</div>;

  return (
    <div className="pt-24 pb-10 container mx-auto px-4 min-h-screen">
      <h2 className="text-3xl font-bold mb-8 text-primary border-l-4 pl-4 flex items-center gap-3">
        <FaHeart className="text-red-500"/> My Favorites List
      </h2>

      {favoriteMovies.length === 0 ? (
        <div className="text-center py-20 dark:text-gray-400 text-lg">
          Your list is empty! Go to "Browse" to add movies.
        </div>
      ) : (
        <div className="grid grid-cols-2 md:grid-cols-4 lg:grid-cols-5 gap-6">
          {favoriteMovies.map(movie => <MovieCard key={movie.id} movie={movie} />)}
        </div>
      )}
    </div>
  );
};

export default MyList;
```

Code Description:-

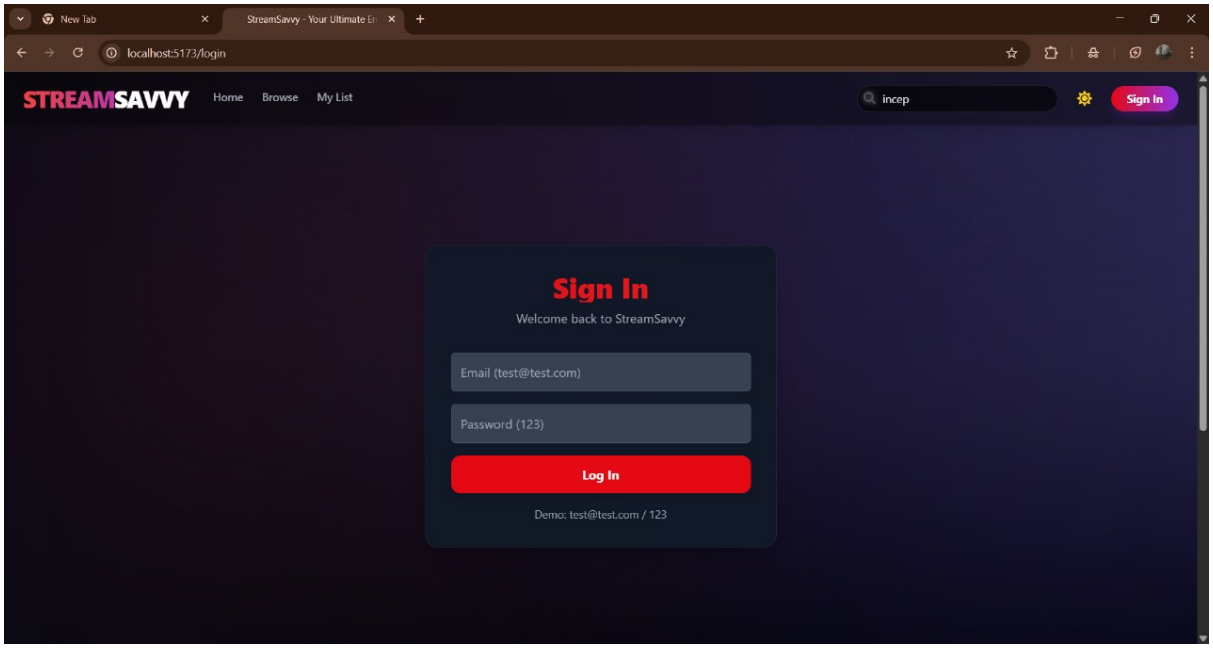
- The code imports React, useState, and useEffect to manage component state and lifecycle.
- It imports two API functions: **getFavorites** and **getMovieById** to fetch favorite movie data from the backend.
- It imports the **MovieCard** component to display each movie on the page.
- toast is used for showing error messages, and FaHeart is used for the heart icon.
- Inside the MyList component, two states are created:
    - favoriteMovies → stores the list of movies
    - loading → shows loading status
- When the component loads, useEffect automatically calls fetchFavorites() once.
- fetchFavorites() first gets the list of favorite movie IDs, then uses Promise.all to fetch the full details of each movie.
- If successful, the movie data is stored in favoriteMovies; if something goes wrong, an error toast appears.
- While loading, the page shows “Loading My List…”.
- If the list is empty, a message tells the user to add movies from the Browse page.
- If movies exist, they are displayed in a grid layout using the MovieCard component.
- Finally, the MyList component is exported so it can be used in routes.

Project Execution:

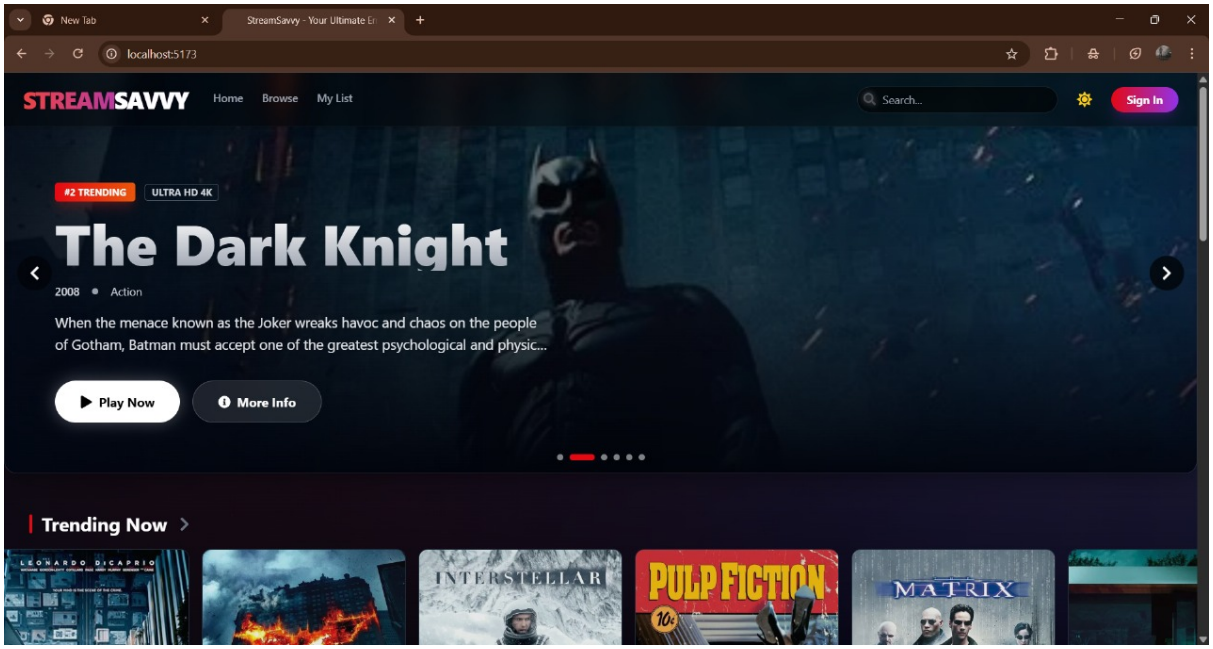
After completing the code, run the react application by using the command “npm start” or “npm run dev” if you are using vite.js

Here are some of the screenshots of the application.

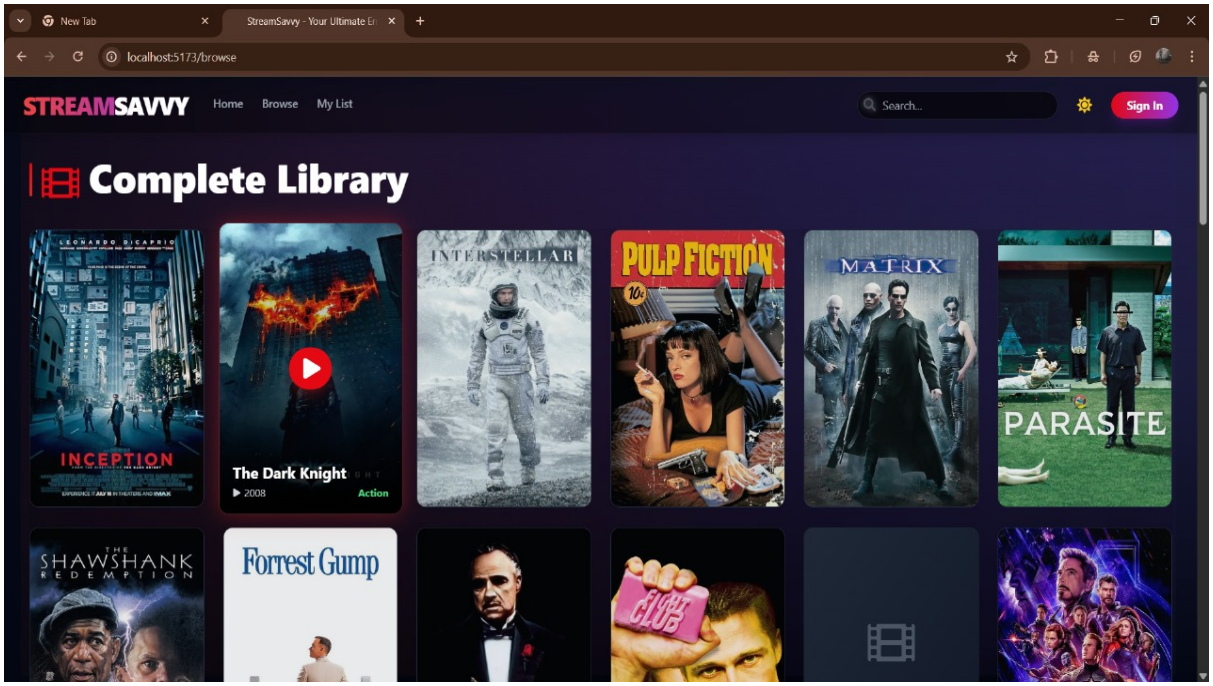
Login page



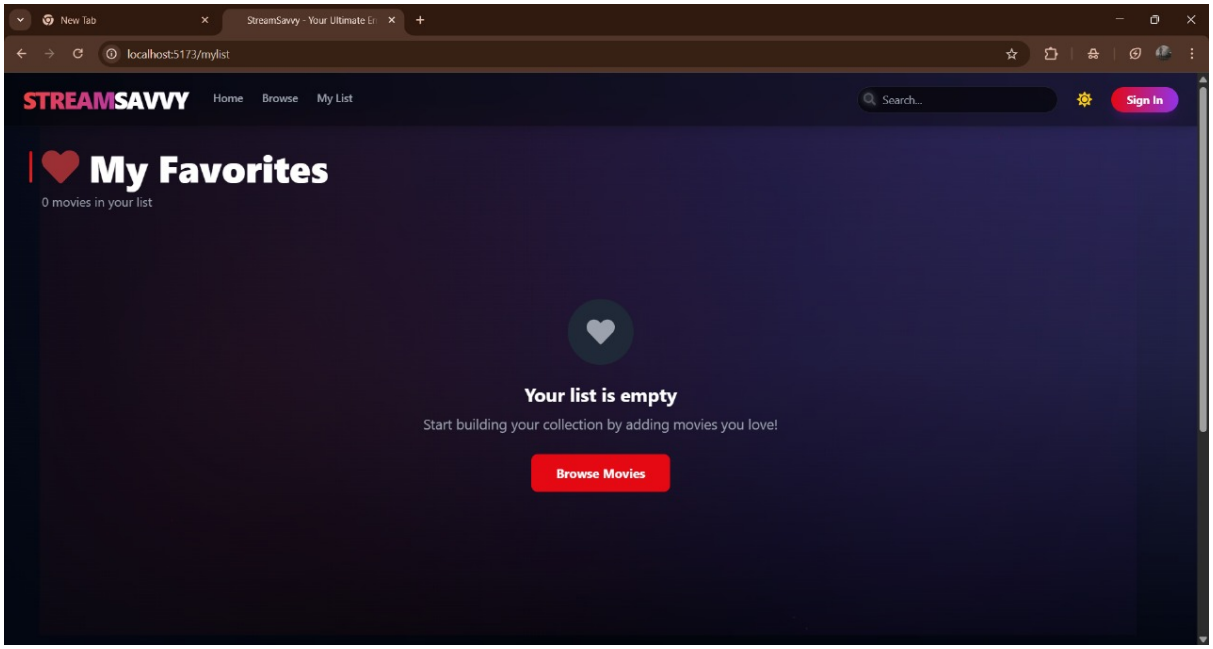
Home Page



Browser Page



My List



Search